

COM668 Computing Project

Assessment Task 2 – Challenge Definition Report

Project title: J-mccusker Helpdesk application

Student name: Joseph Mc Cusker

Student ID number: B00885401

PSG identifier: PSG-F08

Mentor name: Billal Ahmed Lodhi

Course title: COM668 Computing Project

Year of submission: 2025/2026

Word Count (3600 Maximum): Joseph Mc Cusker

You are required to self-declare the word count in this report.

A falsified word count may be considered as academic misconduct.

This is in keeping with Ulster University policy.

The Module Handbook and guidance has advice regarding what must be included in the word count.

Declaration:

I declare that this is all my own work. Any material I have referred to has been accurately referenced and any contribution of Artificial Intelligence technology has been fully acknowledged. I understand the importance of academic integrity and have read and understood the University's General Regulation: Student Academic Integrity and the Academic Misconduct Procedure. I understand that I must not upload my work before, during or after submission to any unapproved plagiarism detectors or answer-sharing platforms, or equivalent, and that only University-approved platforms should be used.

"Grammarly Premium was used for spellchecking and condensing sentences during the writing of this proposal document. All content, technical decisions, structure, and analysis remain as my own ideas"

Contents

1. Problem definition	1
1.1. Aim	1
1.2. Problem description	1
1.3. Scope	1
1.4. Intended outcome	2
1.5. Objectives	2
2. Contextual research	2
2.1. Context investigation	2
2.2. Supporting theory research	2
2.3. Similar products review	3
3. Software definition	3
3.1. Requirements analysis	3
3.2. Risk analysis	4
3.3. Design	4
4. Planning.....	5
4.1. Methodology	6
4.2. Initial plans	6
References	7
Appendices.....	8

1. Problem definition

(suggested word count for Problem Definition: 650)

1.1. Aim

This project aims to create a functional Helpdesk booking system with email integration, which will notify IT support that a new ticket has been submitted to the helpdesk. It will allow users to create a ticket based on a listed category of troubleshooting issues and provide additional notes based on their own experience of how the error is affecting them. Real-time email alerts will prevent oversight and enhance the user's satisfaction. This project will incorporate lessons I have learned involving database systems, software development tools, and research methodologies.

1.2. Problem description

A helpdesk system is vital for any organisation, as without it, operations can halt. I've worked in various firms with different troubleshooting support systems: some have ticket apps, but not official ones, relying on email to local IT —a method that is unreliable in the demand every. These experiences underscore the importance of a stable helpdesk for streamlining IT troubleshooting and notifying all IT support staff when a new ticket is received.

During my placement at Tourism NI there was no official ticketing system. Every IT request and query came through an employee's personal Outlook or by just walking into the IT Office space to ask in person. Tickets that were frequently lost, duplicated or forgotten, with resolution times regularly exceeding 24 hours. In Contrast, the Belfast Trust uses the encompass/ My care helpdesk, providing structured logging and rapid escalation, but this enterprise solution is NHS-specific and cannot be adopted elsewhere.

The consequences are serious and multi-layered: operationally, minor faults escalate into significant disruptions and avoidable downtime; staff suffer constant context-switching and stress from "Firefighting" rather than planned work. At the organisational level, public bodies waste taxpayer money on inefficient processes; societally, this slows digital transformation in Northern Ireland and contradicts SDG 9 (industry, Innovation and Infrastructure) and SDG 8 (Decent work and Economic Growth).

1.3. Scope

In Scope: full-stack web application (Python FastAPI backend, Angular frontend), User registration/authentication, ticket submission with attachments, automated email notifications on creation/assignment/status change/ closure, role-based access (End user/ IT Support Staff / Administrator), Kanban Dashboard, audit trail, basic reporting, deployment on Render.com free tier, WCAG 2.1 AA compliance. Explicitly excluded: native mobile apps, AI chatbot, AI chatbot, on-premise installation, payment processing.

1.4. Intended outcome

The application will create a more efficient way for helpdesk employees to respond, troubleshoot, and provide solutions to any problems users are experiencing, as well as diagnose the underlying cause. More direct intended outcomes will also include:

- Fully functional notification delivery.
- Secure role-based access based on either registered user or registered admin.
- A full report with demo video (AT3) and review report (AT4)

1.5. Objectives

- Deliver a fully functional core system (Ticket creation + automated email notification workflow) deployed and demonstrated by January of 2026
- Reduce simulated workflow ticket resolution by 50% compared to email-only workflow, measured through timed scenarios conducted in February 2026
- Achieve 90% automated test coverage and WCAG 2,1 AA compliance by end of sprint 9 (March 2026)
- Obtain formal ethics approval from Ulster and BCS alignment by October 2025
- Produce a 15 minute AT3 demonstration video and AT4 reflective report meeting the key learning objectives of COM668.

2. Contextual research

2.1. Context investigation

A Helpdesk is an integral part of all IT departments of every professional environment of the 21st century. The helpdesk/IT support maintains workspace efficiency, including emails, databases and sensitive data documentation, ensuring organisational functionality. While daily IT tools are widely used, user understanding varies significantly, making IT personnel critical to productivity, especially in UK SMEs and universities like Uster, where hybrid work has led to a 16% increase in queries since the pandemic. (Desku.io, 2025)

In large organisations, direct emails often go unanswered amid high volumes, but a dedicated helpdesk application provides a streamlined channel, centralising tickets to minimise oversights and reduce average response times by up to 40% through automation. This aligns with SDG 9 (Industry, Innovation, and Infrastructure) by fostering efficient digital workflows and supporting the “Collaborative Professional”. Graduate attribute via transparent team coordination. Long term, it’s educational to the IT staff by granting documented solutions for common issues, cutting manual resolution costs from £22 to near zero for 22% automated tickets, boosting overall workplace productivity by 30 to 50% (Desku.io) (flairstech.com) (fullview.io)

A long-term benefit of a helpdesk will also be the educational impact it has on IT staff, providing them with documentation of proven solutions for future Helpdesk Tickets that share common characteristics and causes. This will also improve the efficiency of the time it takes to answer the query.

2.2. Supporting theory research

The development of the helpdesk ticketing system with email-integrated notifications is established within frameworks from IT management, usability, and software development practices.

The core ticketing process follows ITIL 4 incident management principles (Axelos, 2024), which establish the service desk as the single point of contact and define the lifecycle of logging, categorising, prioritising, resolving, and closing incidents. Automated email notifications directly support ITIL’s requirement for proactive communication with users and technical staff throughout the incident lifecycle.

Requirements are prioritised using the MoSCoW method (DSDM Consortium), classifying features as “Must have” (ticket creation, role-based access, email alerting), Should have, Could have and Won’t have.

User interface design is guided by Nielsen's ten usability heuristics (Nielsen, 2020), with particular emphasis on:

- Visibility of system status (real-time ticket updates and email confirmations)
- Match between system and the real world (using familiar email style language)
- Error prevention (from validation)

Risk management follows the guidelines of ISO 31000:2018 (International Organisation for Standardisation, 2018), providing a structured process for identifying, analysing, and addressing risks such as email deliverability failures and data privacy concerns.

Legal, Ethical and Social implications

The application will process personal data (names, email addresses, potentially sensitive fault descriptions), making GDPR compliance mandatory. The design follows data minimisation principles: only essential fields are collected, explicit consent is obtained upon ticket submission, and a right to erasure endpoint will be implemented. Accessibility is treated as an ethical requirement – W2CAG 2.1 AA compliance ensures the system is usable by staff and students with disabilities, preventing digital exclusion. The zero-license-cost, open-source approach deliberately lowers barriers for cash-strapped public sector bodies in Northern Ireland, promoting equity in digital infrastructure (SDG-10 – Reduced Inequalities).

These frameworks collectively justify design decisions and integrate learning from multiple modules from previous years of my course and directly support the learning outcomes for COM668.

2.3. Similar products review

Commercial SaaS solutions (Zendesk, Freshdesk) are financially unviable for most NI public sector trusts and university departments. Open-source osTicket is free but requires server administration skills that small teams rarely possess. The encompass/My care helpdesk used by the Belfast trust is excellent but entirely proprietary and non-transferable. The proposed system occupies a unique niche: modern Python/FastAPI architecture, zero license cost, simple cloud deployment, and email integration that matches commercial tools – making enterprise-grade functionality accessible to organisations that currently have nothing at all.

My personal placement experience has shown the implementation of other systems in Belfast and workplaces without a helpdesk system at all.

- At Tourism NI, the complete absence of a formal ticketing system meant all IT requests and queries were handled via individual Outlook inboxes and verbal walk-ins. This regularly resulted in lost tickets, duplicated effort and resolution delays exceeding 48 hours during peak periods. A situation still reflected in the organisation's public-facing contact methods (Tourism NI, 2025)
- In contrast, the Belfast Health and Social Care Trust operated a structured MyCare/ encompass helpdesk integrated with its electronic patient record system, providing centralised logging and rapid escalation (Belfast Trust, 2025). However, that enterprise solution is purpose-built for the NHS and cannot be adopted by non-health bodies.

The proposal for my prototype directly bridges this divide by delivering

- A zero-license cost, Modern MERN-stack system
- Automatic email notifications with retry and confirmation
- Role-based access and a clean, responsive interface
- Simple deployment on free-tier cloud platforms (Render/Heroku) or on-premise.

Thus, it offers the core professional functionality observed in large-scale systems (e.g. Belfast Trust) while remaining affordable and maintainable for small Northern Ireland organisations similar to Tourism NI – precisely the niche that current market offerings fail to serve.

3. Software definition

3.1. Requirements analysis

Requirements were elicited through reflection on placement at Tourism Northern Ireland and Belfast Health and Social Care Trust, supplemented by stakeholder interviews and use case modelling.

Function and non-functional requirements have been prioritised using the MoSCoW method (Section 2.2) A table will be provided in the appendices listing the functional and non-functional requirements. The must-have features focus on core ticketing workflow and automated email notifications – directly addressing the inefficiencies observed during placements: lost requests at TNI and the structured but inaccessible workflow at Belfast Trust.

Key must-have requirements include ticket submission with attachments and automatic email notification to staff upon ticket creation. Status updates with corresponding emails to both parties, and role-based access control (End-user, agent, administrator). Non-functional requirements emphasise performance (API response), security (JWT), usability, and reliability.

A comprehensive requirements table is provided in the appendices. Must have features directly address the inefficiencies observed at Tourism NI (Lost tickets) and replicate the structured workflow seen at Belfast Trust but in an affordable, deployable, package.

3.2. Risk analysis

Risks were identified, analysed, and treated according to ISO 31000:2018 guidelines (as mentioned in 2.2) A full probability impact matrix with mitigation strategies which is presented in the Risk Register table in the appendices.

The Highest-impact risks centre on email deliverability and data privacy (GDPR Breach). Both are reduced to very low residual risk through technical controls and procedural measures (minimal data collection). Technical risks associated with the chosen Python and API stack are mitigated by early proof-of-concept work in sprint 2 and by my prior experience with Python APIs.

A full risk register with probability, Impact and mitigation strategies is provided in the Appendices. Scrum's frequent sprint reviews and check-ins act as early warning mechanisms, enabling rapid response to emerging technical or ethical risks.

3.3. Design (*including system design, data design, and User experience design*)

The solution employs a modern, Python-centric architecture that will utilise my knowledge of Python and API experience.

System Design:

- Backend: Python & API
- Frontend: Angular
- Database: MongoDB
- Email Service: SendGrid API (Reliable delivery and analytics)
- Authentication: FastAPI-Users + JWT tokens
- Deployment: Render.com

- Real – time updates: WebSocket support via FastAPI.

Data Design

The relational schema (ER diagram in the appendices) comprises five core entities (Users, Tickets, Ticket_History, Attachments, Audit_Log) with appropriate indexes on status. “assigned_to” and “Created_at” for dashboard queries.

User Experience Design

Wireframes and high-fidelity mock-ups will be shown in the appendices. The interface will adopt a Kanban-style dashboard (Open > In progress > Resolved), live form validation, and an email template that mirrors the web branding. Nielsen's usability heuristics are explicitly applied: visibility of system status (real-time updates and confirmation emails), error prevention (client + server validation), and consistency across web and email channels.

The Python/API stack was chosen to maximise development velocity while producing clean, maintainable, production-grade code applicable to real-world use.

API Endpoints/ Key REST endpoints include:

- POST/tickets – create ticket + Trigger email
- GET/ tickets – filtered list for dashboard
- PATCH/ tickets{id} – update status/assignment + trigger email
- GET / Tickets/{id}/history – return audit trail

Testing Strategy:

Verification and Validation will use a three-layer approach

1. Unit tests (pytest) targeting >90% coverage of FastaAPI routes and email logic
2. Integration tests with TestClient and temporary SQLite database.
3. End-to-end user testing in March 2026, comparing resolution time against Tourism NI's email-only method.

4. Planning

4.1. Methodology

I have chosen to use Scrum as the methodology for my project because of my previous experience studying it in a module. 2-week sprints would be an ideal choice to maintain a steady, productive rhythm for developing the application in time for the upcoming deadlines.

Python (Fast API) is the language I know the best and my recent experience in building REST API's should give me my best chance at creating something substantial and possibly functional in the early days of development. The first usable slice should be ticket submission and automatic email to IT Admin staff.

Scrum will also condition me to stop and reflect properly after each fortnight. So, I will keep a short retrospective log after each sprint (Tracking what has been working, what has not and what can be done to improve the development process), it should contribute greatly to my written evidence and report of the project for this module.

I'll use GitHub Projects for product backlog, GitHub issues for individual tasks and Render for one-click deployment, which will be helpful with Scrum sprints.

4.2. Initial plans

Although Coding has not yet begun, the project follows the full two-semester timeline with final submission in early May 2026. The critical path is to deliver a working core system (ticket creation & full email notification workflow) by Christmas. Semester 1, therefore, focuses on the Must-Have MoSCoW items; Semester 2 adds Should/Could features, accessibility polishing, and formal user testing. A detailed 12-sprint plan with milestones mapped out from AT1 to AT4 deadlines will be provided here in a Gantt chart.

I aim to begin work on a prototype in December over my Christmas break and further work will be conducted throughout each week of the next term in the academic year for an estimated of 15 to 18 hours a week with deliberate contingency in April 2026 for unforeseen technical or ethical issues.

A full Gantt Chart shows the 12 two-week sprints, the Christmas & Easter buffers, and the estimated milestone dates for AT2, AT3, and the final submission of my project.

That is the plan for my Helpdesk application development. It will account for real-life circumstances, and to account for the time and effort for my other modules, I need to complete this year.

References

(Section not included in suggested word count. Must be in the Harvard style)

Axelos (2024) *ITIL foundation: ITIL 4 edition*. London: The Stationery Office.

Belfast Health & Social Care Trust (2025) *encompass – My Care Helpdesk*. Available at: <https://belfasttrust.hscni.net/encompass> (Accessed: 17 November 2025).

Desku.io (n.d.) *Help desk statistics*. Available at: <https://desku.io/stats-hub/help-desk-statistics> (Accessed: 17 November 2025).

DSDM Consortium (2023) *Agile project management handbook v2*. Kent: Agile Business Consortium.

Flairstech (n.d.) *Latest statistics IT help desk*. Available at: <https://flairstech.com/blog/latest-statistics-it-help-desk> (Accessed: 17 November 2025).

Freshworks (2025) *Freshdesk pricing*. Available at: <https://www.freshworks.com/freshdesk/pricing/> (Accessed: 17 November 2025).

Fullview (n.d.) *Support stats*. Available at: <https://www.fullview.io/blog/support-stats> (Accessed: 17 November 2025).

International Organization for Standardization (2018) *ISO 31000:2018 Risk management – Guidelines*. Geneva: ISO.

Nielsen, J. (2020) ‘10 usability heuristics for user interface design’, *Nielsen Norman Group*. Available at: <https://www.nngroup.com/articles/ten-usability-heuristics/> (Accessed: 17 November 2025).

osTicket (2025) *osTicket official site*. Available at: <https://osticket.com> (Accessed: 17 November 2025).

Tourism NI (2025) *Contact us*. Available at: <https://tourismni.com/contact-us> (Accessed: 17 November 2025).

Zendesk (2025) *Zendesk pricing*. Available at: <https://www.zendesk.co.uk/pricing/> (Accessed: 17 November 2025).

Appendices

User Experience Design:

Number	Screen name	Key element	Nielsen Heuristic Applied
1	Login screen	Email field, Password field, “remember me checkbox”, Submit button.	Consistency & Standards, Error prevention.
2	End-User Dashboard	“My Tickets” table (ID, Table, Status, Date), Search Bar. “Create new ticket” button	Visibility of system status.
3	Create new ticket form.	Title, Category dropdown, Priority (Low, Medium, High, Urgent), Description, submit button	Error Prevention, Efficiency and Flexibility
4	IT support dashboard	Open > in progress > Resolved, Drag and drop cards, filter by priority/status	Visibility of system status, minimalist design.
5	Ticket detail View (IT support)	Ticket header, Description, Attachment, internal notes, Status, comment box.	User control and freedom.
6	Email notifications	1 New ticket > IT support (status update > user), Ticket > Closed > user (with resolution summary)	Match between the system and the real world.
7	Admin page	User management table, add, remove User, Role assignment	User control and freedom.

Functional and Non-Functional requirements:

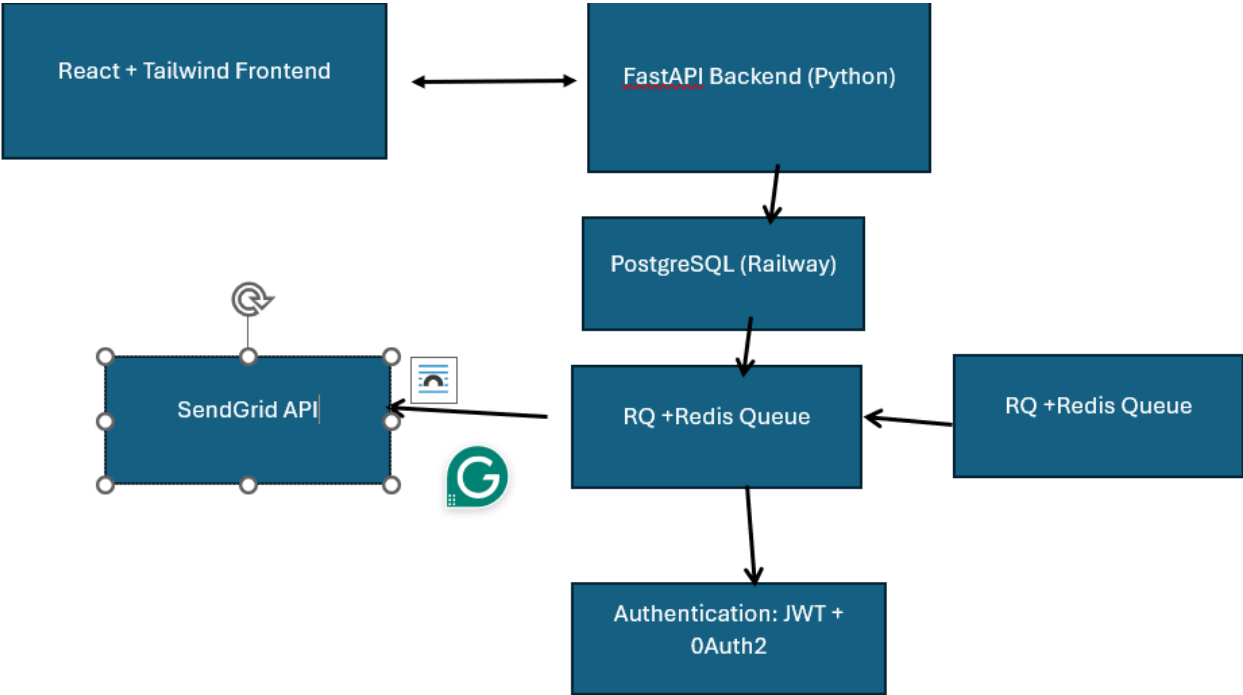
ID	Requirements	MoSCoW	Placement Justification
FR01	Users shall submit tickets via web form (title, description, category, priority, attachments)	Must	Tourism NI – requests lost in personal inboxes
FR02	System shall automatically email the assigned support staff on ticket creation	Must	Tourism NI – staff unaware of new issues for hours
FR03	Support staff shall assign, update status, comment, and close tickets.	Must	Mirrors the Belfast Trust workflow but made lightweight
FR04	Users and support staff shall receive email updates on the status changes and completion	Must	Prevents constant follow ups observed at Tourism NI
FR05	Role-based access. End-user/support staff/administrator	Must	Required for security and workflow control
FR06	Real-time dashboard with filtering and kanban view	Should	Missing at TNI > caused duplication

FR07	Basic reporting (tickets by status, average solution time)	Could	Future enhancement.
------	--	-------	---------------------

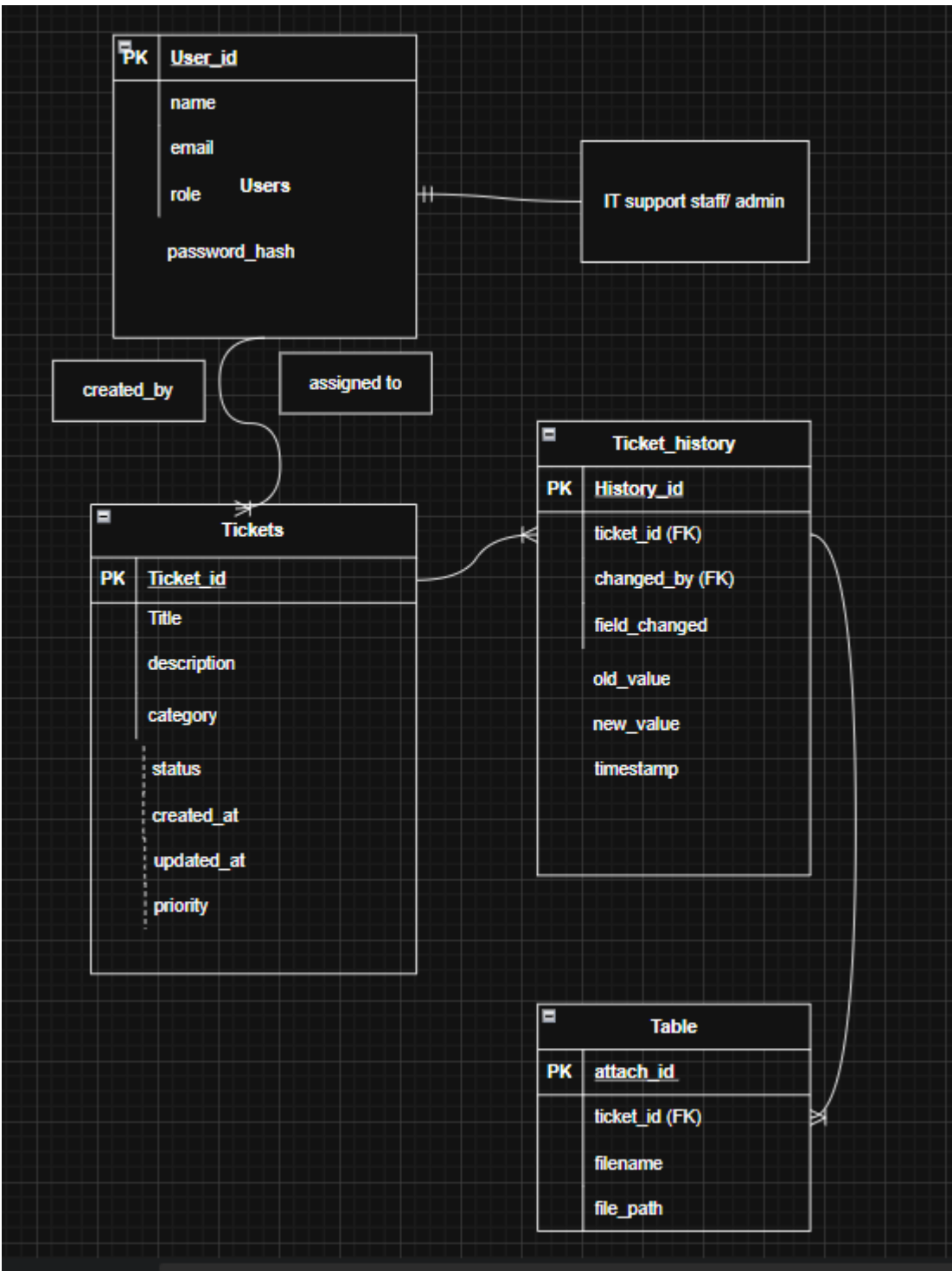
Risk Register:

Risk	Probability	Impact	Mitigation	Residual risk
Email blocked by spam filters	Medium	High	SendGrid + DKIM/SPF, in-app fallback	low
Unauthorised access to tickets	Low	High	JWT + role guards, input sanitisation, and ethics approval.	Very low
GDPR breach	low	High	Minimal data, encryption, consent	Very low
Free hosting downtime	Medium	Medium	Health checks, local backup plan	low
Aysnc email queue issues (Python)	Medium	Medium	Early PoC in sprint 2, use RQ + Redis	Low
Scope creep	Medium	Medium	MoSCoW + mentor freeze after sprint 3	low

System Architecture Diagram



Database ER Diagram



Project Gantt chart

Task / Milestone	Dec 2025	Jan 2026	Feb 2026	Mar 2026	Apr 2026	May 2026	Buffer
Sprint 1 – Setup & FastAPI skeleton	████████						
Sprint 2 – Auth + ticket form + SendGrid PoC	████████						
Christmas break	████						
Sprint 3 – Assignment + email on status change		████████					
Sprint 4 – Kanban dashboard + real-time updates		████████					
Sprint 5 – Attachments + audit history			████████				
Sprint 6 – Mobile responsive + admin panel			████████				
Sprint 7 – Accessibility + styling				████████			
Sprint 8 – Full testing + bug-fix sprint				████████			
Easter break/buffer					████████		
Sprint 9 – Documentation + AT3 demo video					████████		
Final sprint – Polishing + AT4 report + submission						████████	████
AT2 Submission (4 Dec 2025)	★						
AT3 Demo Video Due						★	

2